Thought for a few seconds

Of course — this is your `main.c` file. I'll explain it **line by line in simple viva style**.

#include <stdio.h>

This includes the **standard input/output library**.
It is used for functions like `printf()`.

#include <stdlib.h>

This includes the **standard utility library**.
It is needed here for functions like `malloc()` and `free()`.

#include <pthread.h>

This includes the **POSIX thread library**.
It is required because your program creates and manages multiple threads using `pthread_create()`, `pthread_join()`, etc.

#include <unistd.h>

This provides access to system-level functions like `usleep()`.
Here it is used to add a small delay before shutting down.

#include "pm.h"

This includes your own header file, which contains:

- function declarations like `pm_init()`, `pm_ps()`, `execute_script()`

- shared variables like `table_lock`, `monitor_running`
- type definitions related to the process manager

Structure definition

typedef struct {

  int id;

  char* filename;

} thread_arg_t;

This defines a structure named `thread_arg_t`.

It contains:

- `id` → the thread number
- `filename` → the script file that thread will execute

Why needed?
Because `pthread_create()` can pass only **one argument** to the worker function, so you pack both values into one struct.

Worker thread function

void* worker(void* arg) {

This is the function each thread will run.
`pthread_create()` requires the thread function to have this form:

- returns `void*`
- takes `void*` argument

```
thread_arg_t* t = (thread_arg_t*)arg;
```

The generic `void*` argument is typecast back into a `thread_arg_t*` pointer.
Now the thread can access its `id` and `filename`.

```
current_thread_id = t->id;
```

This stores the thread's ID in `current_thread_id`.
That variable is declared as thread-local in `pm.h`, so each thread keeps its own separate copy.
This is useful for logging messages like "Thread 0 calls pm_fork 1."

```
execute_script(t->filename);
```

This tells the thread to read and execute commands from its assigned script file.
For example, a thread may execute commands like:

- `fork 1`
- `wait 1 -1`
- `exit 2 10`

```
free(t);
```

The argument struct was created dynamically using `malloc()` in `main()`.
After the thread finishes using it, the memory is released with `free()`.

---

return NULL;

}

The thread function ends and returns `NULL`.

Main function

int main(int argc, char *argv[]) {

This is the main entry point of the program.

- `argc` = number of command-line arguments
- `argv` = array of argument strings

Example run:

./pm_sim thread0.txt thread1.txt thread2.txt

Then:

- `argc = 4`
- `argv[0] = "./pm_sim"`
- `argv[1] = "thread0.txt"` and so on

---

if (argc < 2) {

This checks whether the user gave at least one script file.

Why `< 2`?
Because `argv[0]` is the program name, so at least one extra argument is needed.

```
    printf("Usage: ./pm_sim thread0.txt thread1.txt ...\n");
```

If no script file is given, the program prints the correct usage format.

```
    return 1;
```

Program exits with status `1`, meaning abnormal or error termination.

```
  }
```

End of the `if` block.

```
  pm_init();
```

This initializes the process manager.

Inside `pm_init()`:

- the initial process with PID 1 is created
- the monitor thread is started
- synchronization setup is completed

```
  int n = argc - 1;
```

The number of worker threads is calculated.

Why `argc - 1`?
Because the first argument is the program name, and the rest are script files.

So if user provides 4 script files, then `n = 4`.

```
pthread_t* threads = malloc(n * sizeof(pthread_t));
```

This dynamically allocates memory for an array of thread handles.

- `pthread_t` is the data type for a thread
- one entry is needed for each worker thread

This allows later joining all threads.

Creating threads

```
for (int i = 0; i < n; i++) {
```

This loop creates one worker thread for each input script file.

```
    thread_arg_t* arg = malloc(sizeof(thread_arg_t));
```

Memory is allocated for the thread's argument struct.

Each thread gets its own separate argument object.

```
    arg->id = i;
```

Assigns the thread ID.

So first thread gets ID 0, next gets 1, and so on.

```
    arg->filename = argv[i + 1];
```

Assigns the script filename for that thread.

Why `i + 1`?
Because `argv[0]` is the program name, so actual script files start from `argv[1]`.

```
    pthread_create(&threads[i], NULL, worker, arg);
```

This creates a new thread.

Parameters:

- `&threads[i]` → where the thread handle will be stored
- `NULL` → default thread attributes
- `worker` → function the thread will run
- `arg` → argument passed to that function

So each thread starts running `worker(arg)`.

```
  }
```

End of thread creation loop.

Waiting for all threads to finish

```
  for (int i = 0; i < n; i++) {
```

This second loop waits for all worker threads to finish.

```
    pthread_join(threads[i], NULL);
```

`pthread_join()` blocks the main thread until `threads[i]` completes.

This ensures:

- all scripts finish execution
- process simulation is complete before shutdown

```
}
```

End of join loop.

Final delay

```
/* Final delay to ensure monitor prints the very last change */
```

This is a comment explaining why the next line exists.

```
usleep(100000);
```

The main thread sleeps for 100000 microseconds = 100 milliseconds.

Reason:
the monitor thread writes snapshots asynchronously, so this short pause gives it time to print the final update before shutdown.

Stopping the monitor thread

```
pthread_mutex_lock(&table_lock);
```

Locks the shared process table before changing shared monitor variables.
This prevents race conditions.

```
monitor_running = 0;
```

This tells the monitor thread to stop its loop.

```
pthread_cond_signal(&cond_mon); // Wake monitor one last time to exit
```

The monitor thread may be sleeping on the condition variable `cond_mon`.
This signal wakes it up so it can notice `monitor_running = 0` and exit cleanly.

```
pthread_mutex_unlock(&table_lock);
```

Unlocks the mutex after updating shared data.

Final output

```
printf("Simulation complete.");
```

Prints a completion message.

```
pm_ps(); // Final display for user
```

Calls `pm_ps()` to print the final process table to the terminal.
This shows the last state of all non-terminated processes.

Cleanup

```
free(threads);
```

Releases the memory allocated for the thread array.

```
return 0;
```

Ends the program successfully.
`0` means normal termination.

```
}
```

End of `main()`.

Viva-friendly summary

You can describe this file like this:

“This `main.c` file is responsible for starting the simulation. It checks input files, initializes the process manager, creates one worker thread per script file, waits for all threads to finish, then stops the monitor thread and prints the final process table.”

Most important viva points from this file

They may ask these directly:

Why use `pthread_create()`?
To run multiple script files concurrently.

Why use a struct for thread arguments?
Because thread functions accept only one argument, so both thread ID and filename are packed together.

Why `pthread_join()`?
To ensure main waits until all worker threads finish.

Why `usleep(100000)`?
To give the monitor thread time to log the last state change.

Why lock `table_lock` before changing `monitor_running`?
Because it is shared data, and mutex prevents race conditions.

Why signal `cond_mon` before exit?
To wake the monitor thread if it is sleeping, so it can terminate properly.

If you want, I'll now explain `pm.h` and `pm.c` in the same **line-by-line viva style**.

Pasted text(4).txt

Document

explain each line

Alright — now this is the **main logic file (`pm.c`)**, and it's the **most important part for viva**. I'll explain it **line by line in simple, clear language**.

🔷 Headers

#include <stdio.h>

Used for input/output (e.g., `printf`, `fprintf`).

#include <stdlib.h>

Used for memory and utilities (`malloc`, etc.).

#include <string.h>

Used for string operations (`sprintf`, etc.).

#include <unistd.h>

Used for system calls like `usleep()`.

#include "pm.h"

Includes your custom header:

- PCB structure
- function declarations
- shared variables

🔷 Global Variables

pcb_t table[MAX_PROC];

Array of PCBs → stores all processes.

---

int proc_count = 0, next_pid = 2, monitor_running = 1, monitor_ready = 0;

- `proc_count` → number of processes
- `next_pid` → next process ID (starts from 2 because PID 1 is init)

- `monitor_running` → controls monitor loop
- `monitor_ready` → ensures monitor starts before others

---

char action_message[256] = {0};

Stores messages like:
👉 “Thread 1 calls pm_fork 2”

__thread int current_thread_id = -1;

Thread-local variable:

- Each thread has its own copy
- Used for logging

🔷 Synchronization Objects

pthread_mutex_t table_lock = PTHREAD_MUTEX_INITIALIZER;

Mutex → protects process table (avoids race condition)

---

pthread_cond_t cond_wait = PTHREAD_COND_INITIALIZER;

Used for `wait()` (parent waits for child)

---

pthread_cond_t cond_mon = PTHREAD_COND_INITIALIZER;

Used to wake monitor thread (no polling)

---

pthread_cond_t cond_mon_done = PTHREAD_COND_INITIALIZER;

Used to sync monitor completion

🔷 Helper Function

int find_index(int pid)

Finds index of process in table.

---

for (int i = 0; i < proc_count; i++)

Loops through process table.

---

if (table[i].pid == pid) return i;

If PID matches → return index.

---

return -1;

If not found → return -1.

🔷 Printing Functions

print_row()

const char* states[] = {"RUNNING", "WAITING", "ZOMBIE", "TERMINATED"};

Maps state enum to string.

---

if (p.state == TERMINATED) return;

Skip terminated processes.

---

fprintf(out, "%d\t\t%d\t\t%s\t\t", p.pid, p.ppid, states[p.state]);

Print PID, PPID, state.

---

if (p.state == ZOMBIE) fprintf(out, "%d\n", p.exit_status);

If zombie → print exit status.

---

else fprintf(out, "-\n");

Otherwise → no exit status.

---

print_table()

fprintf(out, "PID\t\tPPID\t\tSTATE\t\tEXIT_STATUS\n");

Prints table header.

---

for (int i = 0; i < proc_count; i++)

Loop through all processes.

---

print_row(out, table[i]);

Print each row.

🔷 Monitor Thread (VERY IMPORTANT)

void* monitor_thread(void* arg)

Special thread → logs process changes.

FILE *fp = fopen("snapshots.txt", "w");

if (fp) fclose(fp);

Creates/clears file.

---

pthread_mutex_lock(&table_lock);

Lock shared data.

FILE *f = fopen("snapshots.txt", "a");

Open file in append mode.

---

fprintf(f, "Initial Process Table\n");

Write initial state.

---

print_table(f);

Print process table.

---

monitor_ready = 1;

pthread_cond_broadcast(&cond_mon_done);

Notify main → monitor is ready.

Main loop

while (monitor_running)

Runs until stopped.

---

pthread_cond_wait(&cond_mon, &table_lock);

Wait for signal (NO polling!)

---

if (!monitor_running && action_message[0] == '\0') break;

Exit condition.

---

if (action_message[0] != '\0')

If there is a message:

---

fprintf(f, "%s\n", action_message);

Write action message.

---

print_table(f);

Write updated table.

---

action_message[0] = '\0';

Clear message.

---

pthread_cond_broadcast(&cond_mon_done);

Notify that logging is done.

---

pthread_mutex_unlock(&table_lock);

return NULL;

Unlock and exit thread.

🔷 pm_init()

pthread_mutex_lock(&table_lock);

Lock table.

---

table[proc_count++] = (pcb_t){1, 0, RUNNING, -1};

Create init process (PID=1).

---

pthread_t mon;

pthread_create(&mon, NULL, monitor_thread, NULL);

Start monitor thread.

---

pthread_detach(mon);

Detach → no need to join.

---

while (!monitor_ready)

Wait until monitor is ready.

---

pthread_cond_wait(&cond_mon_done, &table_lock);

Sleep until signal.

---

pthread_mutex_unlock(&table_lock);

Unlock.

🔷 pm_fork()

pthread_mutex_lock(&table_lock);

Lock table.

---

if (proc_count >= MAX_PROC || find_index(p_pid) == -1)

Check:

- table full OR parent doesn't exist

---

int new_pid = next_pid++;

Generate new PID.

---

table[proc_count++] = (pcb_t){new_pid, p_pid, RUNNING, -1};

Add new process.

---

sprintf(action_message, "Thread %d calls pm_fork %d", current_thread_id, p_pid);

Log message.

---

pthread_cond_signal(&cond_mon);

Wake monitor.

---

pthread_cond_wait(&cond_mon_done, &table_lock);

Wait until monitor finishes.

---

pthread_mutex_unlock(&table_lock);

return new_pid;

Unlock and return new PID.

🔷 pm_exit()

table[i].state = ZOMBIE;

table[i].exit_status = status;

Process becomes zombie.

---

pthread_cond_broadcast(&cond_wait);

Wake waiting parent.

🔷 pm_kill()

Same as exit but:

exit_status = -1;

Means forced kill.

🔷 pm_wait() (VERY IMPORTANT)

while (1)

Infinite loop → waits until condition satisfied.

Check children

if (table[i].ppid == p_pid)

Find child processes.

If no child

return -1;

If zombie found

int status = table[zombie_idx].exit_status;

Get exit status.

---

shift array left

Remove zombie from table.

---

return status;

Return child status.

If child running

table[p_idx].state = WAITING;

Parent becomes waiting.

---

pthread_cond_wait(&cond_wait, &table_lock);

Sleep until child exits.

🔷 pm_ps()

print_table(stdout);

Print table to screen.

🔷 execute_script()

FILE* fp = fopen(filename, "r");

Open script file.

---

while (fgets(line, sizeof(line), fp))

Read line by line.

---

if (sscanf(line, "fork %d", &a) == 1)

Parse command.

Calls:

- `pm_fork()`
- `pm_exit()`
- `pm_wait()`
- `pm_kill()`
- `usleep()` for delay

---

fclose(fp);

Close file.

🎯 FINAL VIVA SUMMARY (VERY IMPORTANT)

Say this confidently:

👉 “This file implements a process management simulator using a PCB table, thread-based execution, mutex for mutual exclusion, and condition variables for synchronization. It simulates fork, wait, exit, and kill, and uses a monitor thread to log state changes without polling.”

🚀 If you want next

I can:
✅ Give **exact viva questions your teacher may ask**
✅ Create **short answers to memorize**
✅ Draw **process lifecycle diagram**

Just tell me 👍

#ifndef PM_H #define PM_H #include <pthread.h> #define MAX_PROC 64 typedef enum { RUNNING, WAITING, ZOMBIE, TERMINATED } state_t; typedef struct { int pid; int ppid; state_t state; int exit_status; } pcb_t; /* Global synchronization objects */ extern pthread_mutex_t table_lock; extern pthread_cond_t cond_wait; extern pthread_cond_t cond_mon; // Mandatory for "No Polling" requirement extern pthread_cond_t cond_mon_done; extern int monitor_running; extern int monitor_ready; extern char action_message[256]; extern __thread int current_thread_id; /* API */ void pm_init(); int pm_fork(int parent_pid); void pm_exit(int pid, int status); int pm_wait(int parent_pid, int child_pid); void pm_kill(int

pid); void pm_ps(); void execute_script(char* filename); #endif explain each line

Perfect — this is your **`pm.h` header file**, and it's very important for viva because it defines the **structure, shared variables, and APIs**.

I'll explain **each line clearly and simply** 👇

🔷 Header Guard (VERY IMPORTANT)

#ifndef PM_H

Checks: *"Is PM_H not defined?"*

👉 Prevents multiple inclusion of the same header file.

#define PM_H

Defines `PM_H`.

👉 After this, if the file is included again, it will be skipped.

🔷 Library Include

#include <pthread.h>

Includes POSIX thread library.

👉 Needed because:

- mutex (`pthread_mutex_t`)
- condition variables (`pthread_cond_t`)

are used in this file.

🔷 Constant Definition

#define MAX_PROC 64

Defines the maximum number of processes.

👉 Your process table can hold **up to 64 processes**.

🔷 Enum (Process States)

typedef enum { RUNNING, WAITING, ZOMBIE, TERMINATED } state_t;

Defines possible process states:

- `RUNNING` → currently executing
- `WAITING` → blocked (waiting for child)
- `ZOMBIE` → finished but not cleaned
- `TERMINATED` → completely removed

👉 This mimics real OS process states.

🔷 PCB Structure (VERY IMPORTANT)

typedef struct {

  int pid;

  int ppid;

```
  state_t state;
  int exit_status;
} pcb_t;
```

This defines the **Process Control Block (PCB)**.

Each process has:

- `pid` → process ID
- `ppid` → parent process ID
- `state` → current state (RUNNING, etc.)
- `exit_status` → exit code (used when ZOMBIE)

👉 This is the **core data structure** of your project.

🔷 Global Synchronization Objects

These variables are declared as `extern`.

👉 Meaning:

- They are **defined in pm.c**
- Used in multiple files

```
extern pthread_mutex_t table_lock;
```

Mutex for protecting the process table.

👉 Prevents **race conditions**.

```
extern pthread_cond_t cond_wait;
```

Condition variable for `wait()`.

👉 Parent process sleeps here until child exits.

---

extern pthread_cond_t cond_mon;

Condition variable for monitor thread.

👉 Used to wake monitor when a change happens
👉 Important for **NO POLLING requirement**

---

extern pthread_cond_t cond_mon_done;

Used to notify:
👉 “Monitor finished logging”

---

extern int monitor_running;

Flag to control monitor thread loop.

👉 If 0 → monitor stops

---

extern int monitor_ready;

Flag to check if monitor thread has started.

👉 Used in `pm_init()` to wait until monitor is ready

---

extern char action_message[256];

Stores messages like:

Thread 1 calls pm_fork 2

👉 Printed by monitor thread

---

extern __thread int current_thread_id;

Thread-local variable.

👉 Each thread has its own copy
👉 Used to identify which thread performed an action

🔷 API Function Declarations

These are function prototypes (declarations).

👉 They tell the compiler:
“these functions exist somewhere (in pm.c)”

---

void pm_init();

Initializes the system:

- creates initial process
- starts monitor thread

---

int pm_fork(int parent_pid);

Creates a child process.

👉 Returns new PID

---

void pm_exit(int pid, int status);

Terminates a process.

👉 Moves it to ZOMBIE state

---

int pm_wait(int parent_pid, int child_pid);

Parent waits for child.

👉 Returns child's exit status

---

void pm_kill(int pid);

Forcefully kills a process.

👉 Sets it to ZOMBIE with status = -1

---

void pm_ps();

Prints process table.

👉 Similar to Linux `ps` command

---

void execute_script(char* filename);

Executes commands from script file.

👉 Used by worker threads

🔷 End of Header Guard

#endif

Ends the `#ifndef PM_H`.

👉 Ensures file is included only once.

🎯 Viva Summary (VERY IMPORTANT)

If examiner asks:

👉 **“What is pm.h?”**

You say:

> “pm.h is the header file that defines the PCB structure, process states, synchronization variables, and function prototypes used for process management simulation.”

🔥 Most Asked Viva Questions from This File

❓ **Why use header guard?**

👉 To prevent multiple inclusion errors.

❓ **What is PCB?**

👉 Structure storing process info (pid, state, etc.)

❓ **Why extern variables?**

👉 To share variables across multiple files.

❓ **Why use __thread?**

👉 To give each thread its own copy of a variable.

❓ **What is MAX_PROC?**

👉 Maximum number of processes allowed.

If you want, I can now:
✅ Take viva like a teacher (real questions)
✅ Give you short answers to memorize
✅ Make diagrams for revision

Just tell me 👍

main(1).c

File

pm(1).c

File

pm(1).h

File

what these 3 files do , for which code cause which output explain each and every thing

Thought for a couple of seconds

Yes — these **3 files work together as one complete process-management simulator**.

Think of them like this:

- `pm.h` = the **blueprint / definitions**
- `pm.c` = the **engine / logic**
- `main.c` = the **driver / controller**

Now I'll explain:

1. what each file does,
2. how they connect,
3. which code causes which output,
4. the full flow from input to output.

Citations are from the files you uploaded: `main.c` , `pm.c` , `pm.h` .

---

1. What each of the 3 files does

`pm.h` — header file

This file contains the **shared definitions** needed by both `main.c` and `pm.c`.

It defines:

- maximum process limit: `MAX_PROC 64`
- process states: `RUNNING`, `WAITING`, `ZOMBIE`, `TERMINATED`
- PCB structure `pcb_t`
- shared synchronization variables like mutex and condition variables
- function declarations like `pm_init()`, `pm_fork()`, `pm_wait()`, `pm_exit()`, `pm_kill()`, `pm_ps()`, `execute_script()`

So this file answers:
“What data exists?” and **“What functions are available?”**

`pm.c` — implementation file

This is the **main logic** of the simulator.

It actually implements:

- the process table
- the monitor thread
- `fork`, `exit`, `wait`, `kill`
- printing the process table
- executing commands from script files

So this file answers:
“How does the simulator work internally?”

`main.c` — controller file

This is the program entry point.

It:

- checks command-line arguments
- initializes the simulator using `pm_init()`
- creates worker threads
- assigns one script file to each worker thread
- waits for all threads to finish
- stops the monitor
- prints the final process table

So this file answers:
“How does the whole program start and run?”

---

2. Full relationship among the 3 files

Step-by-step connection

Step 1: `main.c` starts program

The program begins from:

int main(int argc, char *argv[])

This is where execution begins. `main.c` reads filenames passed in command line and starts the simulation.

Step 2: `main.c` uses `pm.h`

`main.c` includes:

#include "pm.h"

That allows `main.c` to know:

- what `pm_init()` is
- what `execute_script()` is
- what `table_lock`, `monitor_running`, `cond_mon` are

Without `pm.h`, `main.c` would not know about those definitions.

Step 3: `main.c` calls functions implemented in `pm.c`

Examples:

- `pm_init();`

- `execute_script(t->filename);`
- `pm_ps();`

These are declared in `pm.h` and defined in `pm.c`.

So the flow is:

main.c → uses declarations from pm.h → calls implementations in pm.c

---

3. What exactly happens when the program runs

Suppose the program is run like this:

./pm_sim thread0.txt thread1.txt thread2.txt thread3.txt

Then the flow is:

In `main.c`

A. argument check

This code checks whether any script file was given:

```
if (argc < 2) {
  printf("Usage: ./pm_sim thread0.txt thread1.txt ...\n");
  return 1;
}
```

Output caused by this code:

If no input files are given, this line prints:

Usage: ./pm_sim thread0.txt thread1.txt ...

So:

- **cause** = `argc < 2`
- **output** = usage message

In `main.c`

B. initialize simulator

This line starts the system:

pm_init();

That jumps to `pm.c` and performs initialization.

---

4. What `pm_init()` does and what output it causes

Inside `pm.c`, `pm_init()` does this:

```
pthread_mutex_lock(&table_lock);

table[proc_count++] = (pcb_t){1, 0, RUNNING, -1};

pthread_t mon;

pthread_create(&mon, NULL, monitor_thread, NULL);

pthread_detach(mon);

while (!monitor_ready) {

  pthread_cond_wait(&cond_mon_done, &table_lock);
```

}

pthread_mutex_unlock(&table_lock);

Meaning:

1. lock the process table
2. create first process:
    - PID = 1
    - PPID = 0
    - state = RUNNING
3. create monitor thread
4. wait until monitor is ready
5. unlock table

Output caused by `pm_init()`

Not directly to terminal.
But it causes the monitor to write the **initial process table** to `snapshots.txt`.

That happens inside `monitor_thread()`:

fprintf(f, "Initial Process Table\n");

print_table(f);

Output in file:

Initial Process Table

PID PPID STATE EXIT_STATUS

---

1 0 RUNNING -

So:

- **cause** = `pm_init()` creates PID 1 and starts monitor

- **output** = initial table written to `snapshots.txt`

---

5. What the monitor thread does

This is one of the most important parts.

The function is:

void* monitor_thread(void* arg)

Its job is:

- create/clear `snapshots.txt`
- write initial table
- sleep until something changes
- whenever process table changes, write:
    - action message
    - new process table

Why is it important?

Because it implements **“No Polling”**.

It waits here:

pthread_cond_wait(&cond_mon, &table_lock);

This means:

- monitor does not continuously check
- it sleeps until another thread signals it

Output caused by monitor thread

Every update in `snapshots.txt` is caused by the monitor thread.

So:

- `pm_fork()` changes table → monitor prints fork result
- `pm_exit()` changes table → monitor prints zombie state
- `pm_wait()` removes child → monitor prints cleaned table
- `pm_kill()` changes state → monitor prints kill result

---

6. What the worker thread does

In `main.c`, each worker thread runs:

```
void* worker(void* arg) {
  thread_arg_t* t = (thread_arg_t*)arg;
  current_thread_id = t->id;
  execute_script(t->filename);
  free(t);
  return NULL;
}
```

Meaning:

- read thread id
- store it in `current_thread_id`
- execute that thread's command file
- free memory
- terminate thread

Why `current_thread_id` matters

Because the message:

```
sprintf(action_message, "Thread %d calls pm_fork %d",
current_thread_id, p_pid);
```

uses it to print which thread did the action.

So:

- **cause** = `current_thread_id = t->id`
- **output** = monitor logs correct thread number

---

7. Process table and PCB

The process table is:

pcb_t table[MAX_PROC];

Each entry is:

```
typedef struct {
  int pid;
  int ppid;
  state_t state;
  int exit_status;
} pcb_t;
```

This means every process stores:

- its PID
- its parent PID
- current state
- exit status

Example

If process 2 is created by process 1 and exits with 10, the PCB might become:

- pid = 2
- ppid = 1
- state = ZOMBIE
- exit_status = 10

That directly causes this row in output:

2 1 ZOMBIE 10

---

8. Which code causes which output

Now the important part.

A. Output: `Thread X calls pm_fork Y`

This output is caused by `pm_fork()`:

sprintf(action_message, "Thread %d calls pm_fork %d", current_thread_id, p_pid);

pthread_cond_signal(&cond_mon);

pthread_cond_wait(&cond_mon_done, &table_lock);

Meaning:

1. message stored in `action_message`
2. monitor is signaled
3. monitor writes it to file
4. caller waits until monitor finishes

Example output:

Thread 0 calls pm_fork 1

Why this happened:

Because thread 0 executed `fork 1`, which came from a script file processed by `execute_script()`.

B. Output: new process row after fork

This line creates the child:

table[proc_count++] = (pcb_t){new_pid, p_pid, RUNNING, -1};

So after fork, the monitor prints a new row like:

2 1 RUNNING -

Cause:

- `pm_fork(1)` created process 2
- state is RUNNING
- exit status is not applicable yet, so `-` printed

Why `-` appears

Because `print_row()` says:

if (p.state == ZOMBIE) fprintf(out, "%d\n", p.exit_status);

else fprintf(out, "-\n");

So only zombie processes show real exit status; others show `-`.

C. Output: `Thread X calls pm_exit pid status`

This is caused by `pm_exit()`:

table[i].state = ZOMBIE;

table[i].exit_status = status;

sprintf(action_message, "Thread %d calls pm_exit %d %d", current_thread_id, pid, status);

pthread_cond_signal(&cond_mon);

Example output:

Thread 2 calls pm_exit 2 10

What changed in table:

Process 2 becomes:

2 1 ZOMBIE 10

So:

- **cause** = `pm_exit(2, 10)`
- **output** = action message + table row with zombie and exit status 10

D. Output: zombie process row

Again caused by:

table[i].state = ZOMBIE;

table[i].exit_status = status;

and then `print_row()` prints exit status if state is zombie.

That is why after exit, the process is not removed immediately. It stays in table as zombie until parent waits.

E. Output: `Thread X calls pm_wait parent child`

This output comes from `pm_wait()` when it finds a zombie child:

```
sprintf(action_message, "Thread %d calls pm_wait %d %d",
current_thread_id, p_pid, c_pid);

pthread_cond_signal(&cond_mon);

pthread_cond_wait(&cond_mon_done, &table_lock);
```

Example output:

```
Thread 0 calls pm_wait 1 -1
```

What it means:

Thread 0 called `pm_wait()` for parent PID 1 and any child (`-1` means any child).

F. Why a process disappears after `pm_wait()`

This code removes the zombie child:

```
for (int i = zombie_idx; i < proc_count - 1; i++) {

  table[i] = table[i+1];

}

proc_count--;
```

This shifts the array left and reduces process count.
So the zombie process is deleted from the process table.

Output effect

Before wait:

1 0 RUNNING -

2 1 ZOMBIE 10

3 1 RUNNING -

After wait:

1 0 RUNNING -

3 1 RUNNING -

Cause:

- `pm_wait()` found zombie child 2
- it removed it from `table[]`
- monitor printed updated table

G. Output: WAITING state

This is caused by `pm_wait()` when the parent has children, but none are zombie yet:

```
if (table[p_idx].state != WAITING) {
  table[p_idx].state = WAITING;
}
pthread_cond_wait(&cond_wait, &table_lock);
```

Meaning:

- parent cannot continue yet
- parent becomes WAITING
- parent sleeps until child exits or is killed

In your current code the logging of “blocks in pm_wait” is commented out, so WAITING may happen internally even if there is no explicit line saying that in `snapshots.txt`.

These commented lines show that:

// sprintf(action_message, "Thread %d blocks in pm_wait", current_thread_id);

// pthread_cond_signal(&cond_mon);

// pthread_cond_wait(&cond_mon_done, &table_lock);

So the state can change to WAITING, but you chose not to print that event message.

H. Output: `Thread X calls pm_kill pid`

This comes from:

table[i].state = ZOMBIE;

table[i].exit_status = -1;

sprintf(action_message, "Thread %d calls pm_kill %d", current_thread_id, pid);

Meaning:

- killed process becomes zombie
- exit status becomes `-1` to indicate forced termination

Output:

Thread 1 calls pm_kill 4

and process row becomes zombie with exit `-1`.

I. Output printed on terminal at the end

At the end of `main.c`:

```
printf("Simulation complete.");

pm_ps();
```

Cause:

- all worker threads have finished
- monitor is told to stop
- final process table is printed to terminal

Output:

```
Simulation complete.

PID       PPID       STATE       EXIT_STATUS

...
```

The actual table depends on which processes remain at the end.

---

9. How `execute_script()` converts text commands into output

This function is extremely important:

```
void execute_script(char* filename) {

  FILE* fp = fopen(filename, "r");

  if (!fp) return;

  char line[128];
```

```
  int a, b;
  while (fgets(line, sizeof(line), fp)) {
    if (sscanf(line, "fork %d", &a) == 1) pm_fork(a);
    else if (sscanf(line, "exit %d %d", &a, &b) == 2) pm_exit(a, b);
    else if (sscanf(line, "wait %d %d", &a, &b) == 2) pm_wait(a, b);
    else if (sscanf(line, "kill %d", &a) == 1) pm_kill(a);
    else if (sscanf(line, "sleep %d", &a) == 1) usleep(a * 1000);
  }
  fclose(fp);
}
```

What it does

Reads a script line by line and matches command patterns:

- `fork n` → call `pm_fork(n)`
- `exit a b` → call `pm_exit(a, b)`
- `wait a b` → call `pm_wait(a, b)`
- `kill a` → call `pm_kill(a)`
- `sleep a` → wait for `a` milliseconds

So the script files are the **input instructions**, and `pm.c` is the logic that turns them into process table changes and output.

---

10. Why `main.c` creates multiple threads

In `main.c`:

pthread_create(&threads[i], NULL, worker, arg);

One worker thread is created for each script file.

Meaning

If you pass 4 script files, 4 worker threads run concurrently.

Each thread executes its own script, so actions can interleave depending on `sleep` timing.

That is why output lines such as:

- `Thread 0 calls pm_fork 1`
- `Thread 1 calls pm_fork 1`
- `Thread 2 calls pm_exit 2 10`

appear in a particular order based on sleep and scheduling, not simply file order.

---

11. Why mutex and condition variables are needed

Mutex

The process table is shared by many threads.
Without mutex, two threads might change it at the same time and corrupt it.

That is why many functions start with:

pthread_mutex_lock(&table_lock);

and end with:

pthread_mutex_unlock(&table_lock);

This guarantees mutual exclusion.

Condition variables

These are used for sleeping and waking.

`cond_mon`

Used to wake the monitor when there is a new action.

`cond_mon_done`

Used so the caller can wait until the monitor has finished logging.

`cond_wait`

Used so a parent in `pm_wait()` can sleep until child exits or is killed.

---

12. Meaning of each process state

These come from:

typedef enum { RUNNING, WAITING, ZOMBIE, TERMINATED } state_t;

`RUNNING`

Process exists and is active.

`WAITING`

Parent is blocked in `pm_wait()` waiting for child.

`ZOMBIE`

Child has finished, but parent has not collected its status yet.

`TERMINATED`

Defined, but in this implementation you mostly remove processes from the table instead of setting them to TERMINATED. That's why `print_row()` skips TERMINATED rows, but your current flow usually deletes zombies after wait instead of marking them TERMINATED.

---

13. Why `pm.h` is necessary

`pm.h` lets both `main.c` and `pm.c` share the same definitions.

Without it:

- `main.c` would not know the prototypes of `pm_init()`, `pm_ps()`, `execute_script()`
- compiler would not know about shared variables like `table_lock`, `cond_mon`, `monitor_running`
- structure `pcb_t` and states would not be shared cleanly

So `pm.h` is the contract between files.

---

14. Simple "who does what" summary

`pm.h`

Defines:

- states
- PCB

- globals
- function declarations

`pm.c`

Implements:

- process table
- monitor
- fork/exit/wait/kill
- script execution
- printing

`main.c`

Controls:

- startup
- thread creation
- worker execution
- shutdown
- final display

---

15. Full cause-and-effect chain

Here is the complete chain:

1. user runs program

`main.c` starts.

2. `pm_init()` called

`pm.c` creates process 1 and monitor thread.

3. `main.c` creates worker threads

Each thread gets one script file.

4. worker calls `execute_script()`

`pm.c` reads commands line by line.

5. command triggers function

Examples:

- `fork 1` → `pm_fork(1)`
- `exit 2 10` → `pm_exit(2,10)`
- `wait 1 -1` → `pm_wait(1,-1)`

6. function changes process table

Example:

- fork adds row
- exit marks zombie
- wait removes zombie
- kill marks zombie with -1

7. function stores action message

Example:

sprintf(action_message, ...)

8. function signals monitor

pthread_cond_signal(&cond_mon);

9. monitor writes snapshot

Writes action message and current process table to `snapshots.txt`.

10. main waits for all threads, stops monitor, prints final table

At the end, `main.c` prints “Simulation complete.” and final process table.

---

16. Best viva answer you can say

If teacher asks: **“What do these three files do together?”**

Say:

> “These three files together implement a multithreaded process management simulator. `pm.h` provides shared definitions and function declarations, `pm.c` implements the process table, synchronization, monitor thread, and operations like fork, wait, exit, and kill, and `main.c` starts the program, creates worker threads for each script file, and prints the final result.”

---

17. Best viva answer for output mapping

If teacher asks: **“Which code causes which output?”**

Say:

> “The output messages are generated in `pm.c`. Functions like `pm_fork`, `pm_exit`, `pm_wait`, and `pm_kill` update the PCB table, store an action string in `action_message`, and signal the monitor thread through `cond_mon`. The monitor thread then writes that action and the updated process table to `snapshots.txt`. The final terminal output comes from `main.c` calling `printf("Simulation complete.")` and `pm_ps()`.”

---

18. One important subtle point for viva

Your code defines `TERMINATED`, but in practice it rarely uses that state meaningfully.
Instead, when a zombie is collected by `pm_wait()`, the process is removed from the table by shifting array elements left. So the simulator mostly shows:

- RUNNING
- WAITING
- ZOMBIE

This is a good thing to mention if examiner asks about `TERMINATED`.

---

19. Final short revision version

`pm.h`

Shared definitions.

`pm.c`

Actual logic and output generation.

`main.c`

Program entry, thread creation, shutdown.

Output generation

- action message created in `pm.c`
- monitor thread prints snapshot

- final table printed by `main.c`

States

- RUNNING
- WAITING
- ZOMBIE
- TERMINATED

Synchronization

- mutex protects table
- cond vars handle waiting and monitor signaling

I can do one more very useful thing: I can make a **full line-by-line mapping table** like
“this line → this effect → this output” for all three files together.

Member 1 Process Manager Core 1. PCB structure design করা 2. global process table বানানো (max 64 process) 3. initial process create করা (PID=1, PPID=0) 4. process states define করা (RUNNING, BLOCKED, ZOMBIE, TERMINATED/REAPED) 5. helper function লেখা যেমন process find করা, free slot খোঁজা, child add/remove করা, next PID allocate করা 6. pm_fork(parent_pid) implement করা 7. pm_exit(pid, status) implement করা 8. pm_kill(pid) implement করা 9. pm_ps() implement করা 10. state string print করার function লেখা process_manager.h এবং process_manager.c এর core অংশ, process table related সব function working -------- this was my part , now explain this 3 files which include for my parts viva and output

Great — now I understand your role clearly.
You were **Member 1: Process Manager Core**.

So I’ll **focus ONLY on your part** and explain:

1. Which parts of the 3 files belong to your work

2. How your code works internally
3. Which output is generated by your part
4. How to explain this in viva (ready-made answers)

🧠 1. Your Responsibility (Core Idea)

You implemented:

👉 **Process Management System (like OS kernel)**

That includes:

- PCB (process structure)
- Process table
- Process creation (fork)
- Process termination (exit/kill)
- Process waiting
- Process display

📂 2. Where your work exists in the 3 files

🔹 `pm.h` → Your DESIGN (structure + definitions)

From this file:

#define MAX_PROC 64

👉 You defined max 64 processes

```
typedef enum { RUNNING, WAITING, ZOMBIE, TERMINATED }
state_t;
```

👉 You defined process states

```
typedef struct {
  int pid;
  int ppid;
  state_t state;
  int exit_status;
} pcb_t;
```

👉 This is your **PCB design** (very important)

🔹 `pm.c` → Your IMPLEMENTATION (main work)

✔ Global Process Table (your work)

```
pcb_t table[MAX_PROC];
int proc_count = 0, next_pid = 2;
```

👉 You created:

- process table
- process counter
- PID generator

✔ Initial Process (your work)

```
table[proc_count++] = (pcb_t){1, 0, RUNNING, -1};
```

👉 You created:

- PID = 1
- PPID = 0
- state = RUNNING

✔ **Helper Function (your work)**

int find_index(int pid)

👉 Finds process in table

✔ **pm_fork() (your core function)**

table[proc_count++] = (pcb_t){new_pid, p_pid, RUNNING, -1};

👉 Creates child process

✔ **pm_exit()**

table[i].state = ZOMBIE;

table[i].exit_status = status;

👉 Process becomes zombie

✔ **pm_kill()**

table[i].state = ZOMBIE;

table[i].exit_status = -1;

👉 Force kill → zombie with -1

✔ **pm_wait() (very important)**

if (table[i].state == ZOMBIE)

👉 Finds zombie child

---

proc_count--;

👉 Removes child from table

✔ **pm_ps()**

print_table(stdout);

👉 Prints process table

✔ **Printing function (your work)**

const char* states[] = {"RUNNING", "WAITING", "ZOMBIE", "TERMINATED"};

👉 Converts state → string

🔹 `main.c` → NOT your logic (but uses your functions)

pm_init();

execute_script(...)

pm_ps();

👉 `main.c` only calls your functions

🔄 3. How YOUR code produces OUTPUT

Now this is the most important part for viva.

🔹 Step-by-step example

Step 1: Initial process

Your code:

```
table[proc_count++] = (pcb_t){1, 0, RUNNING, -1};
```

Output:

```
1   0   RUNNING   -
```

🔹 Step 2: fork

Script:

```
fork 1
```

Your code:

```
int new_pid = next_pid++;

table[proc_count++] = (pcb_t){new_pid, p_pid, RUNNING, -1};
```

Output:

```
Thread 0 calls pm_fork 1

1   0   RUNNING   -
```

2 1 RUNNING -

👉 Why?

- new process added
- state = RUNNING

🔹 Step 3: exit

Script:

exit 2 10

Your code:

table[i].state = ZOMBIE;

table[i].exit_status = status;

Output:

Thread 2 calls pm_exit 2 10

2 1 ZOMBIE 10

👉 Why?

- process finished
- not removed yet

🔹 Step 4: wait

Script:

wait 1 -1

Your code:

```
if (table[i].state == ZOMBIE)

proc_count--;
```

Output:

```
Thread 0 calls pm_wait 1 -1

1   0   RUNNING   -
```

👉 Why?

- parent collects child
- child removed

🔹 Step 5: kill

Script:

```
kill 3
```

Your code:

```
table[i].state = ZOMBIE;

table[i].exit_status = -1;
```

Output:

```
Thread X calls pm_kill 3

3   ?   ZOMBIE   -1
```

👉 Why?

- forced termination

🔹 Step 6: ps

Your code:

pm_ps();

Output:

Final process table printed on screen.

🧠 4. Key Concepts YOU implemented

Say these in viva:

✔ PCB

Stores process info (pid, ppid, state, exit_status)

✔ Process Table

Array storing all processes

✔ Process Lifecycle

RUNNING → ZOMBIE → (wait) → removed

✔ Fork

Creates child process

✔ Exit

Moves process to ZOMBIE

✔ **Wait**

Removes zombie child

✔ **Kill**

Forcefully creates zombie

🎯 5. EXACT VIVA ANSWERS (for your part)

❓ **What did you implement?**

👉
“I implemented the process manager core including PCB structure, process table, process creation, termination, waiting, and display functions.”

❓ **How does fork work?**

👉
“It creates a new PCB entry with a unique PID, sets parent PID, and initializes state to RUNNING.”

❓ **What happens in exit?**

👉
“The process state changes to ZOMBIE and exit status is stored, but the process is not removed immediately.”

❓ **What is zombie?**

👉
“A terminated process that remains in the process table until the parent calls wait.”

❓ **What does wait do?**

👉
“It finds a zombie child, retrieves its exit status, and removes it from the process table.”

❓ **What is process table?**

👉
“A global array storing all PCB entries.”

❓ **Why MAX_PROC?**

👉
“To limit maximum processes and avoid overflow.”

⚡ Final Simple Explanation (say this)

👉
“My part was the core process manager. I designed the PCB

structure, implemented a global process table, and wrote functions like fork, exit, wait, and kill. These functions update the process table, and based on those updates the system prints the process state changes and final output.”

If you want, I can now:
✅ Give **10 viva questions your teacher will definitely ask**
✅ OR explain **your exact snapshot output line-by-line**

Yes — these **3 files work together as one complete process-management simulator**.

thread0(1).txt

Document

thread1(1).txt

Document

thread2(1).txt

Document

thread3(1).txt

Document

snapshots(1).txt

Document

explain those 3 files with the thread text and the output in the snapshot txt

Thought for a couple of seconds

Yes — now we can explain the **3 code files together with the 4 thread text files and how they produced the exact `snapshots.txt` output**.

I'll do it in the order the program actually runs.

Code files:

- `main.c`
- `pm.c`
- `pm.h`

Thread input files:

- `thread0.txt`
- `thread1.txt`
- `thread2.txt`
- `thread3.txt`

Output file:

- `snapshots.txt`

---

1. What the 3 code files do

`pm.h`

This is the **definition file**. It declares:

- max process count = 64
- process states: `RUNNING`, `WAITING`, `ZOMBIE`, `TERMINATED`
- PCB structure:
  - `pid`
  - `ppid`

    - `state`
    - `exit_status`
- shared synchronization variables
- function declarations like `pm_init`, `pm_fork`, `pm_exit`, `pm_wait`, `pm_kill`, `pm_ps`, `execute_script`

So `pm.h` tells the whole program what a process looks like and which functions exist.

`pm.c`

This is the **main logic file**. It implements:

- global process table
- monitor thread
- helper function `find_index`
- `pm_init`
- `pm_fork`
- `pm_exit`
- `pm_kill`
- `pm_wait`
- `pm_ps`
- `execute_script`

This file is the heart of the simulator.

`main.c`

This is the **controller**. It:

- checks command-line input
- initializes the process manager by calling `pm_init()`

- creates one worker thread for each thread text file
- waits for all worker threads to finish
- stops the monitor thread
- prints final process table

---

2. How the program uses the thread text files

In `main.c`, each input file is given to one worker thread:

pthread_create(&threads[i], NULL, worker, arg);

Inside `worker()`:

current_thread_id = t->id;

execute_script(t->filename);

So each worker thread:

1. stores its ID
2. reads its own text file
3. executes commands line by line

The `execute_script()` function in `pm.c` reads commands and maps them like this:

- `fork X` → `pm_fork(X)`
- `exit A B` → `pm_exit(A, B)`
- `wait A B` → `pm_wait(A, B)`
- `kill X` → `pm_kill(X)`
- `sleep N` → `usleep(N * 1000)`

So the thread text files are the **instructions**, and `pm.c` turns those instructions into process-table changes.

---

3. What each thread file does

`thread0.txt`

Contents:

fork 1

sleep 200

wait 1 -1

Meaning:

1. immediately create a child of process 1
2. sleep for 200 ms
3. then parent 1 waits for any child (`-1` means any child)

`thread1.txt`

Contents:

sleep 50

fork 1

sleep 300

Meaning:

1. wait 50 ms
2. create another child of process 1
3. then sleep 300 ms

`thread2.txt`

Contents:

sleep 150

exit 2 10

Meaning:

1. wait 150 ms
2. process 2 exits with status 10

`thread3.txt`

Contents:

sleep 300

fork 3

Meaning:

1. wait 300 ms
2. create a child of process 3

---

4. How output is generated in `snapshots.txt`

The monitor thread in `pm.c` is responsible for writing `snapshots.txt`.

When functions like `pm_fork`, `pm_exit`, `pm_wait`, `pm_kill` change the process table, they do this pattern:

1. update process table
2. store a text message in `action_message`
3. signal the monitor thread
4. wait until monitor logs the change

Example from `pm_fork`:

sprintf(action_message, "Thread %d calls pm_fork %d", current_thread_id, p_pid);

pthread_cond_signal(&cond_mon);

pthread_cond_wait(&cond_mon_done, &table_lock);

The monitor wakes up and writes:

- the action message
- the updated table

to `snapshots.txt`

---

5. Full step-by-step explanation of the actual snapshot output

Now I'll match the exact snapshot lines with the exact input command and code behavior.

Snapshot 1: Initial Process Table

`snapshots.txt` starts with:

Initial Process Table

PID PPID STATE EXIT_STATUS

---

1 0 RUNNING -

Why this happened

Because `main.c` called:

pm_init();

Then in `pm.c`, `pm_init()` created the initial process:

table[proc_count++] = (pcb_t){1, 0, RUNNING, -1};

Then the monitor thread wrote the initial table to `snapshots.txt`

Meaning

- PID 1 = initial process
- PPID 0 = no parent / root process
- state RUNNING
- exit status `-` because it is not a zombie

Snapshot 2: `Thread 0 calls pm_fork 1`

Then snapshot shows:

Thread 0 calls pm_fork 1

PID PPID STATE EXIT_STATUS

---

1 0 RUNNING -

2 1 RUNNING -

Which thread file caused this

`thread0.txt` starts with:

fork 1

Which code caused it

`execute_script()` read `fork 1` and called:

pm_fork(1);

Inside `pm_fork()`:

- new PID is allocated using `next_pid++`
- child is inserted into table:

table[proc_count++] = (pcb_t){new_pid, p_pid, RUNNING, -1};

- action message is set:

sprintf(action_message, "Thread %d calls pm_fork %d", current_thread_id, p_pid);

Why the table looks like this

Now there are 2 processes:

- process 1 is the parent
- process 2 is the new child of process 1

So output becomes:

- `1 0 RUNNING -`
- `2 1 RUNNING -`

Snapshot 3: `Thread 1 calls pm_fork 1`

Next snapshot shows:

Thread 1 calls pm_fork 1

PID	PPID	STATE	EXIT_STATUS
1	0	RUNNING	-
2	1	RUNNING	-
3	1	RUNNING	-

Which thread file caused this

`thread1.txt` says:

sleep 50

fork 1

Why it happens after thread 0

Because thread 1 first sleeps 50 ms, while thread 0 does `fork 1` immediately.

So the order becomes:

1. thread 0 forks first
2. after 50 ms, thread 1 forks

Which code caused it

Again `execute_script()` called:

pm_fork(1);

This created process 3 as another child of process 1.

Why the table now has 3 rows

- PID 1 = original process
- PID 2 = first child of 1
- PID 3 = second child of 1

Snapshot 4: `Thread 2 calls pm_exit 2 10`

Then snapshot shows:

Thread 2 calls pm_exit 2 10

PID	PPID	STATE	EXIT_STATUS
1	0	RUNNING	-
2	1	ZOMBIE	10
3	1	RUNNING	-

Which thread file caused this

`thread2.txt` says:

sleep 150

exit 2 10

Why it happens here

Because:

- thread 0 already forked at time 0
- thread 1 forked at time 50 ms
- thread 2 waits until 150 ms, then makes process 2 exit

Which code caused it

`execute_script()` read `exit 2 10` and called:

pm_exit(2, 10);

Inside `pm_exit()`:

table[i].state = ZOMBIE;

table[i].exit_status = status;

Then it logged:

sprintf(action_message, "Thread %d calls pm_exit %d %d", current_thread_id, pid, status);

Why process 2 is still shown

Because `exit()` does not remove the process immediately.
It changes the state to `ZOMBIE`.

So:

- process 2 finished
- exit status = 10
- but parent has not collected it yet

That is why snapshot shows:

2	1	ZOMBIE	10

Snapshot 5: `Thread 0 calls pm_wait 1 -1`

Then snapshot shows:

Thread 0 calls pm_wait 1 -1

PID	PPID	STATE	EXIT_STATUS
1	0	RUNNING	-
3	1	RUNNING	-

Which thread file caused this

From `thread0.txt`:

fork 1

sleep 200

wait 1 -1

Thread 0 forked immediately, then slept 200 ms, then executed `wait 1 -1`.

Why it happens after `exit`

Because thread 2 exits process 2 at 150 ms, and thread 0 only does wait at 200 ms.

So by the time thread 0 calls wait, process 2 is already zombie.

Which code caused it

`execute_script()` read:

wait 1 -1

and called:

pm_wait(1, -1);

Inside `pm_wait()`:

- it searches for children of parent 1
- since `-1` means any child, it accepts any child of 1
- it finds process 2 in ZOMBIE state
- it saves exit status
- removes process 2 from the table by shifting the array left:

```
for (int i = zombie_idx; i < proc_count - 1; i++) {

  table[i] = table[i+1];

}
```

proc_count--;

Then it logs:

sprintf(action_message, "Thread %d calls pm_wait %d %d", current_thread_id, p_pid, c_pid);

Why process 2 disappears

Because `wait()` reaps the zombie child.

So after wait:

- process 2 is removed
- only process 1 and process 3 remain

That is exactly why the snapshot becomes:

1	0	RUNNING	-
3	1	RUNNING	-

Snapshot 6: `Thread 3 calls pm_fork 3`

Finally snapshot shows:

Thread 3 calls pm_fork 3

PID	PPID	STATE	EXIT_STATUS
1	0	RUNNING	-
3	1	RUNNING	-
4	3	RUNNING	-

Which thread file caused this

`thread3.txt` says:

sleep 300

fork 3

Why it happens last

Because thread 3 waits 300 ms before doing anything, so its fork happens after all earlier events.

Which code caused it

`execute_script()` called:

pm_fork(3);

Inside `pm_fork()`:

- new pid is allocated → 4
- new process is created as child of process 3:

table[proc_count++] = (pcb_t){new_pid, p_pid, RUNNING, -1};

Why output looks like this

Now:

- process 1 exists
- process 3 is child of 1
- process 4 is child of 3

So snapshot becomes:

1	0	RUNNING	-
3	1	RUNNING	-
4	3	RUNNING	-

---

6. Why there is no WAITING state shown in your snapshot

Your enum includes `WAITING` in `pm.h`

And `pm_wait()` can set a parent to WAITING in `pm.c`:

```
if (table[p_idx].state != WAITING) {
  table[p_idx].state = WAITING;
}
```

But in this particular run, thread 0 calls `wait 1 -1` only after process 2 has already become zombie. So `pm_wait()` finds a zombie immediately and reaps it. It does not need to block for long.

Also, the logging lines for “blocks in pm_wait” are commented out in the code, so even if WAITING happened briefly, your snapshot would not explicitly print that event.

---

7. Final timeline in very simple form

Here is the full event order:

Time 0 ms

- `pm_init()` creates process 1
- thread 0 runs `fork 1`
- process 2 is created

Time 50 ms

- thread 1 runs `fork 1`
- process 3 is created

Time 150 ms

- thread 2 runs `exit 2 10`
- process 2 becomes zombie

Time 200 ms

- thread 0 runs `wait 1 -1`
- parent 1 collects zombie child 2
- process 2 is removed

Time 300 ms

- thread 3 runs `fork 3`
- process 4 is created as child of 3

This exactly matches the order in `snapshots.txt`

---

8. Short viva explanation you can say

You can say this:

> "The three files work together as a process manager simulator. `pm.h` defines PCB, states, and shared declarations. `pm.c` implements the process table and process operations like fork, exit, and wait, and it also runs a monitor thread that writes snapshots. `main.c` starts the system and creates one worker thread per script file. The thread text files provide the commands, and each command changes the process table, which is then written to `snapshots.txt` by the monitor thread."

---

9. Short “which input caused which output” answer

You can say:

> “`thread0.txt` created process 2 and later waited for a zombie child, `thread1.txt` created process 3, `thread2.txt` made process 2 exit with status 10, and `thread3.txt` created process 4 as a child of process 3. These actions were logged into `snapshots.txt` by the monitor thread after each process-table update.”

---

10. Final mapping table

Input file line	Function called	Effect on table	Snapshot result
`thread0: fork 1`	`pm_fork (1)`	create PID 2	`Thread 0 calls pm_fork 1` + PID 2 row
`thread1: fork 1`	`pm_fork (1)`	create PID 3	`Thread 1 calls pm_fork 1` + PID 3 row
`thread2: exit 2 10`	`pm_exit (2,10)`	PID 2 becomes zombie	`Thread 2 calls pm_exit 2 10` + zombie row

`thread0: wait 1 -1`	`pm_wait (1,-1)`	reap zombie PID 2	`Thread 0 calls pm_wait 1 -1` + PID 2 removed
`thread3: fork 3`	`pm_fork (3)`	create PID 4	`Thread 3 calls pm_fork 3` + PID 4 row

If you want, I can turn this into a **Bangla viva script** you can memorize word-for-word.